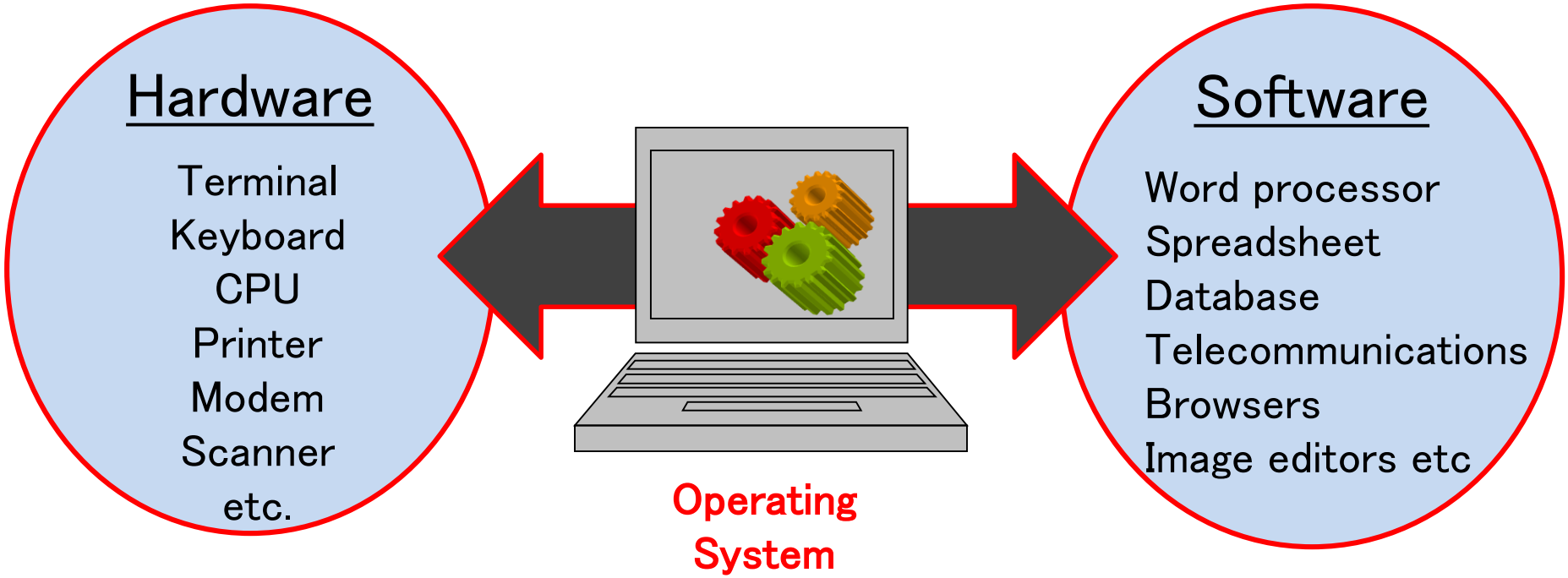
Chapter 1

Introduction to UNIX/Linux Kernel

Dr. Arati Nimgaonkar

Operating System



Controls both hardware and software

Operating System

OS acts as user interface. Performs services on command

- Manages processes…….
- Organizes Files……..
- Controls Access and Assigns Permissions……..
- Allocates Resources…….
- Manages Memory…….

Unix History

- In the 1965, MIT , AT&T Bell Labs, and General Electric designed a operating system called **Multics** (***Multiplexed Information and Computing Service***)

- 1969, Ken Thomson, Dennis Ritchie developed **a new operating system** using Multics which was multiuser (2 user) . Brian Kernighan named this operating system as **Unics** which later gets spelled as **Unix**. It was developed in the assemble language

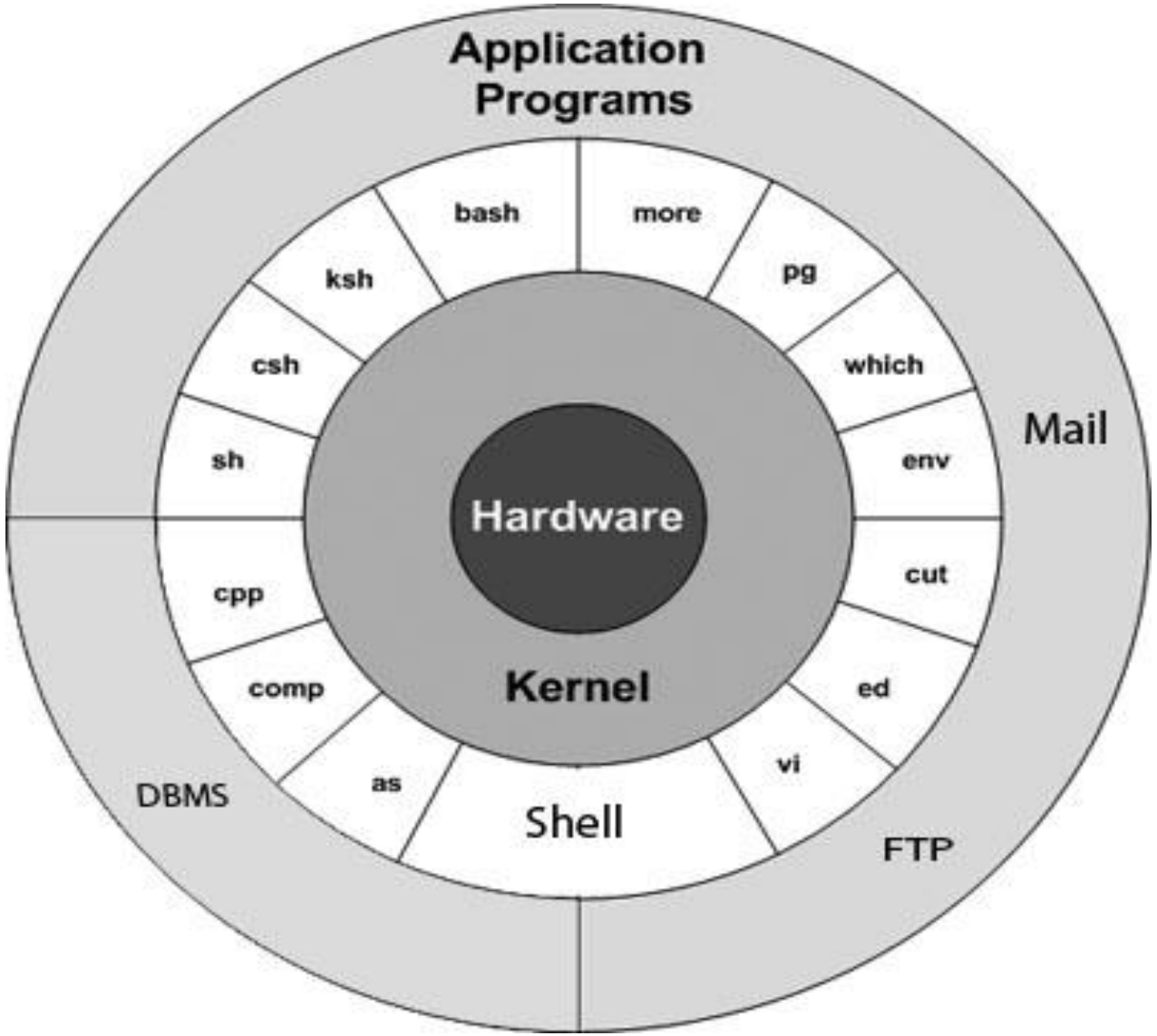
- 1973, Unix was **rewritten in C** language by Ken Thomson and his team. By 1980, around 10 versions of Unix was released with some changes.

- In 1982, AT & T combined all these versions in **Unix System V** which was totally a commercial version that could not serve academic purposes.

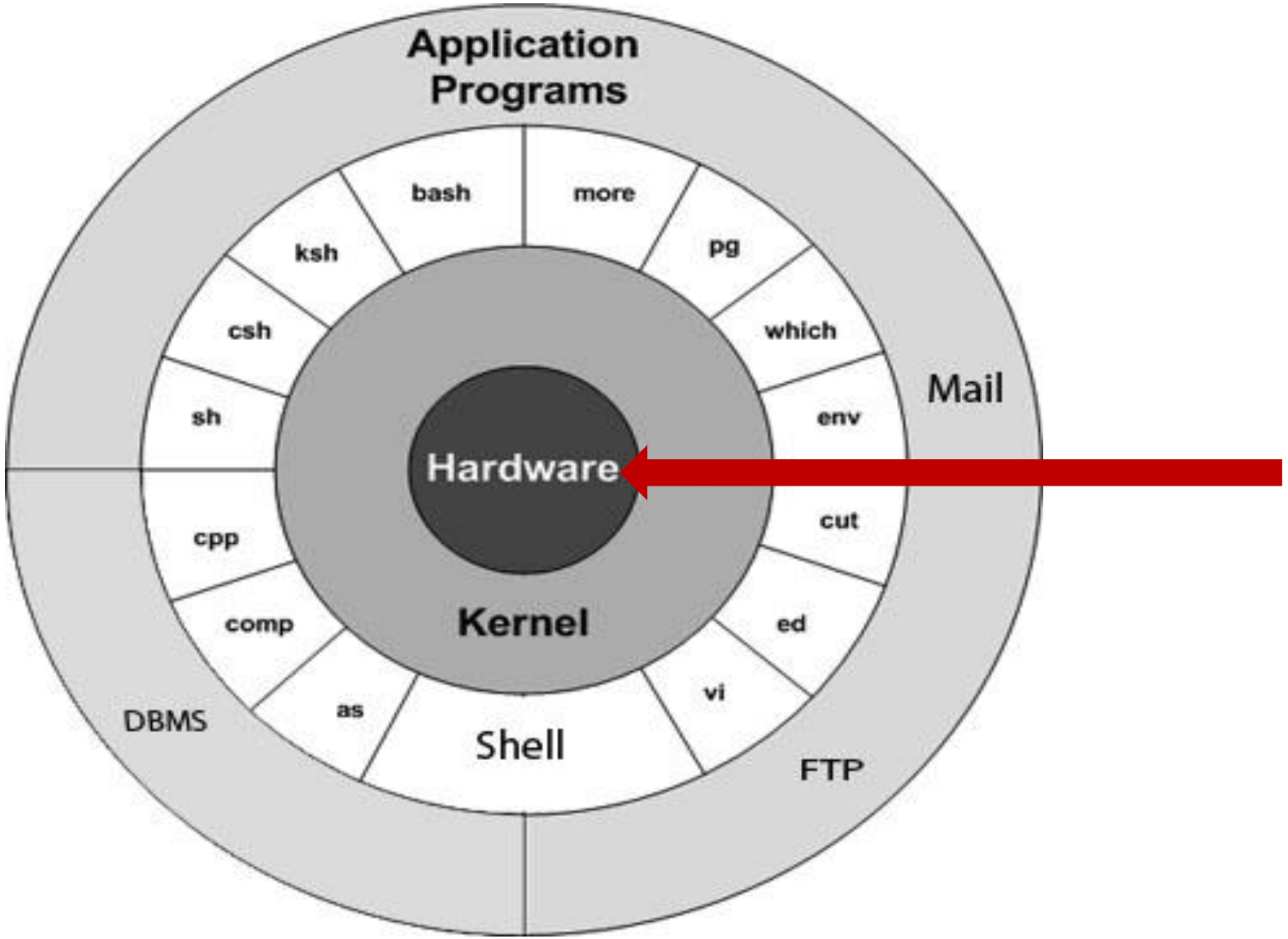
- In early 80's, Berkeley Software Distribution released **BSD Unix** which serves the academic purpose.

- After this, Other companies began to offer commercial versions of the UNIX System for their own mini-computers and workstations

Unix System structure...high level architecture

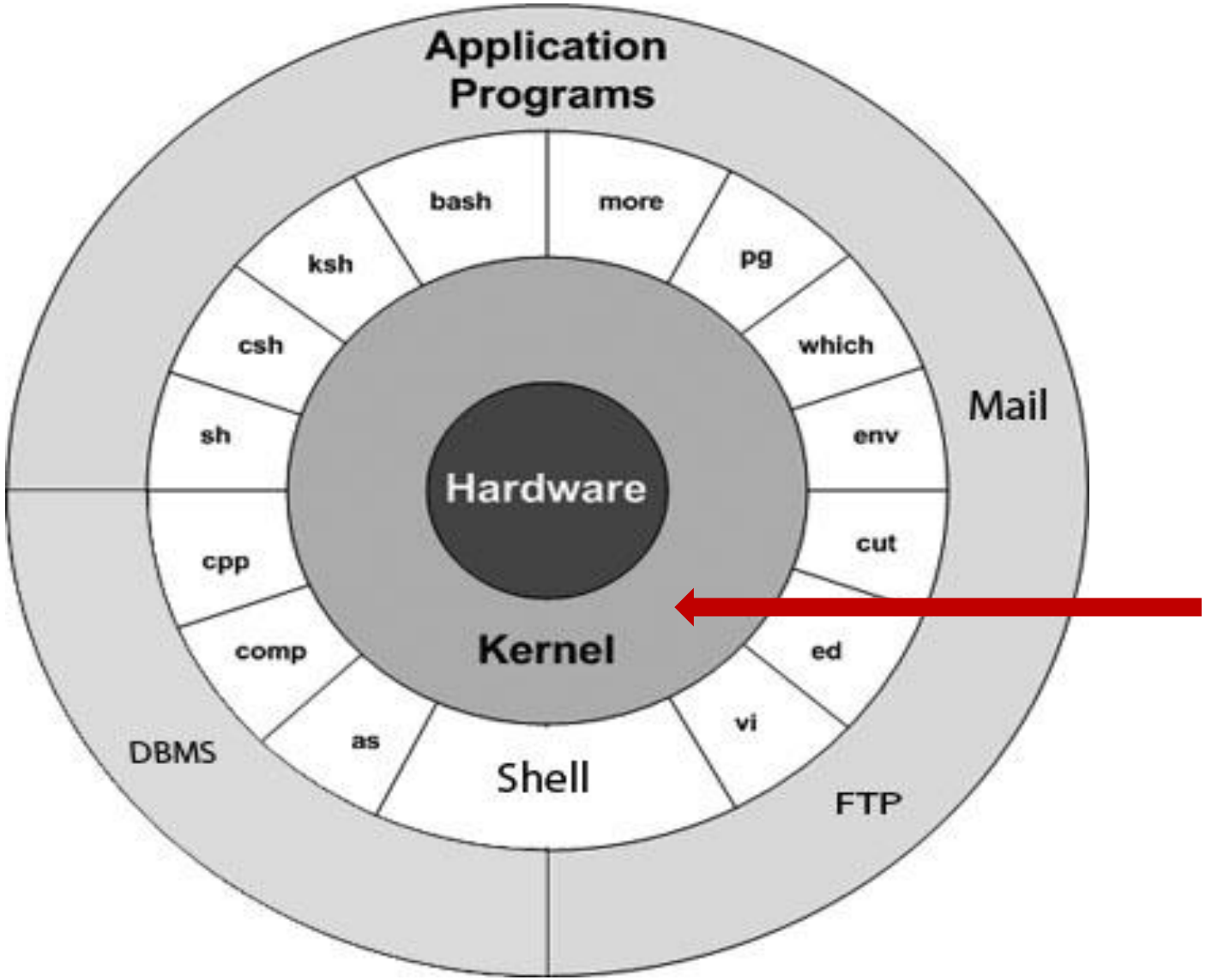


Unix System structure...high level architecture



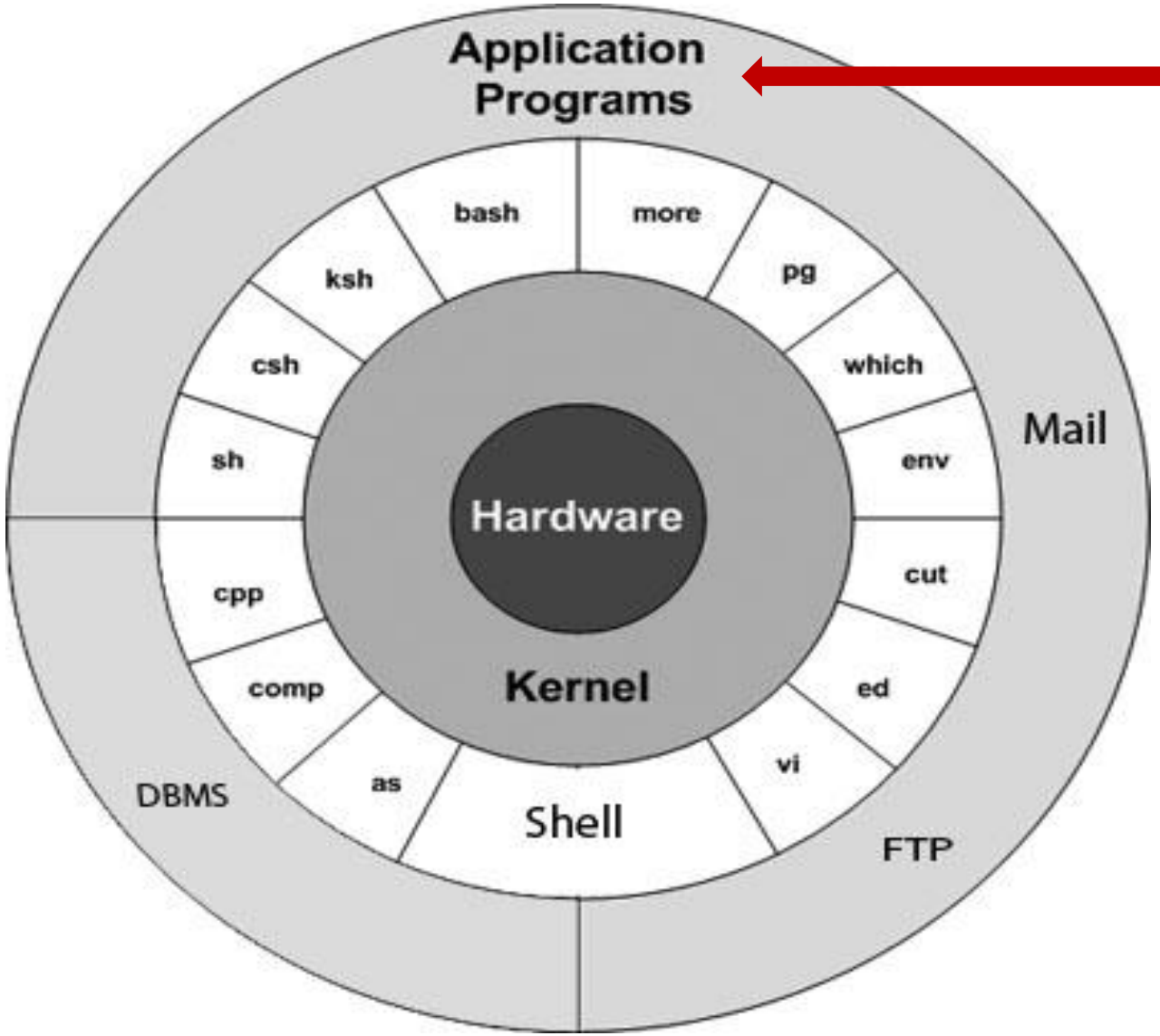
Hardware is the computer components which can be touched and feel.

Unix System structure...high level architecture



Kernel acts as an interface between application programs, utility programs and hardware.

Unix System structure...high level architecture



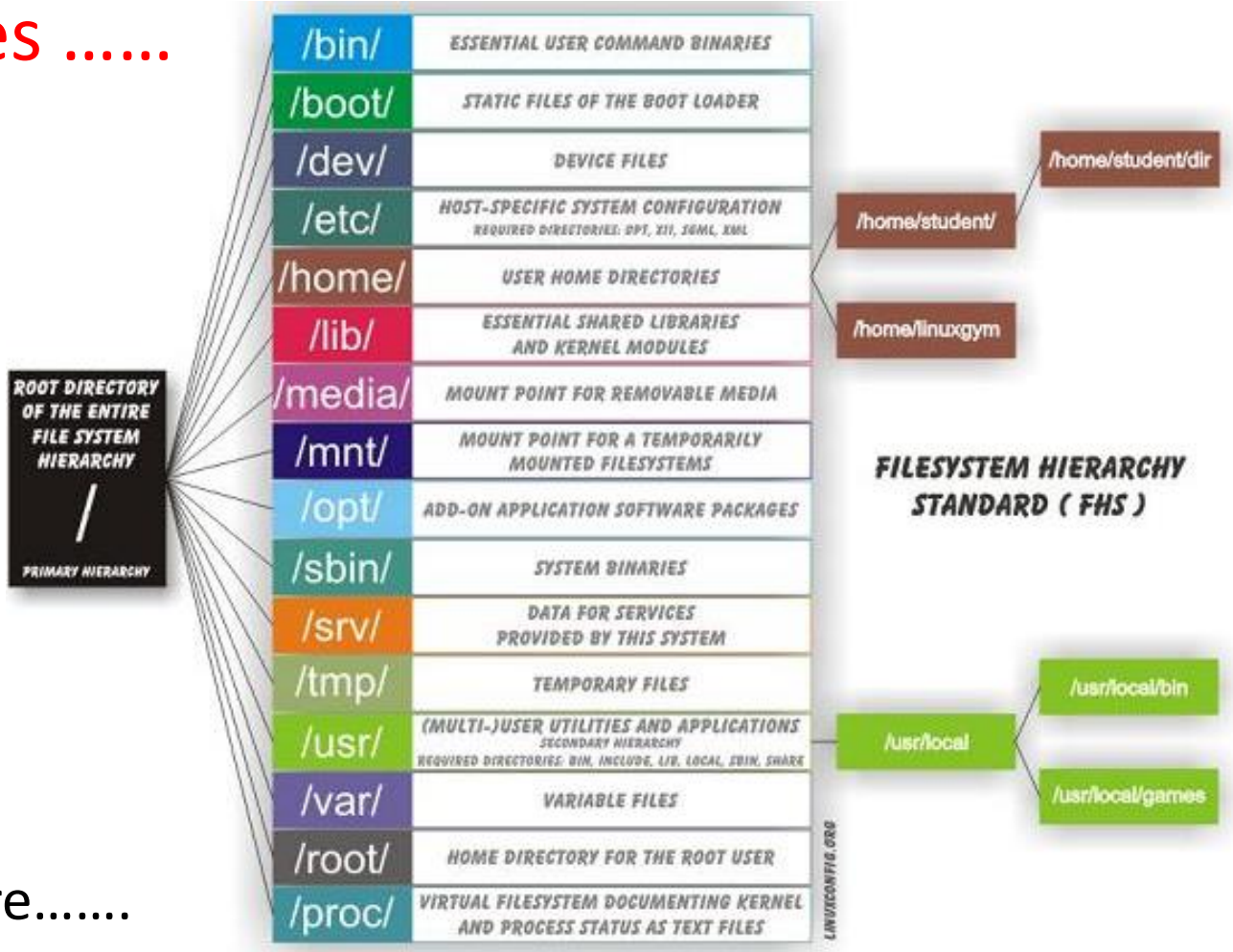
Application program are the user side programs .

UNIX Kernel services

- Manages operations of file subsystem and process subsystem.
- Controls process scheduling and inter process communication.
- Manages the use of peripheral devices.
- Manages memory & swap memory
- Signal handling.

High level features ……

File Tree Structure

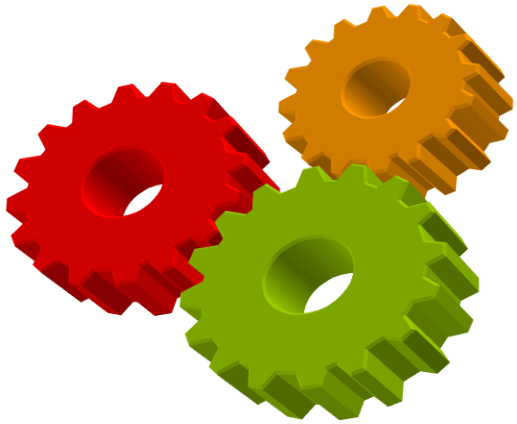


- Hierarchical structure…….
- Consistent treatment to file data……
- The ability to create & delete the files……
- Dynamic growth of a file…….
- Protection of file data……
- Treatment of peripheral devices as a file……

Dr. Arati Manjaramkar

High level features ……

Processing Environment



- Program is an executable file & process is the instance of the program.
- Many processes can execute simultaneously.
- Process is identified by process id i.e. pid
- Processes are created by fork() system call.
- Execution is carried out by exec() system call.

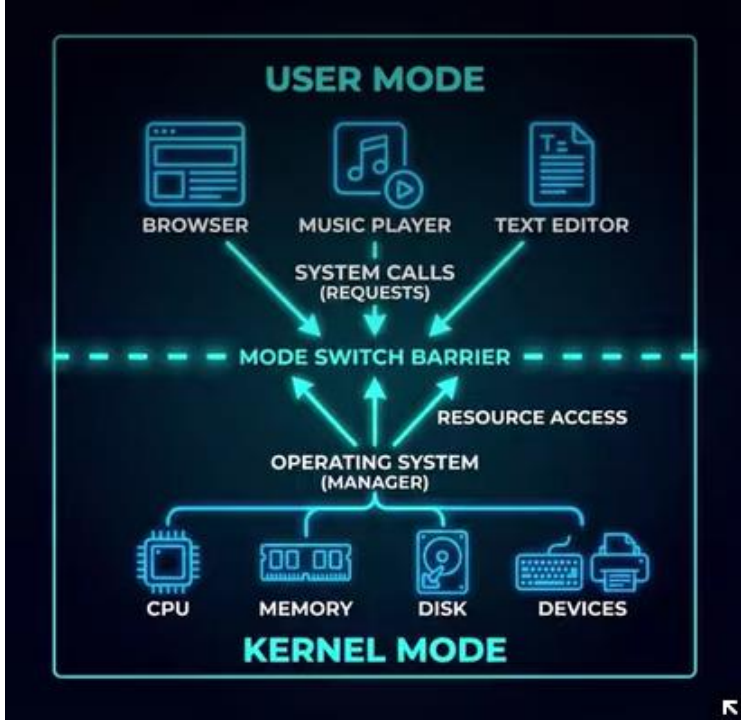
Process Execution levels…..

- Process Execution is divided into 2 levels
  - Kernel level
  - User level

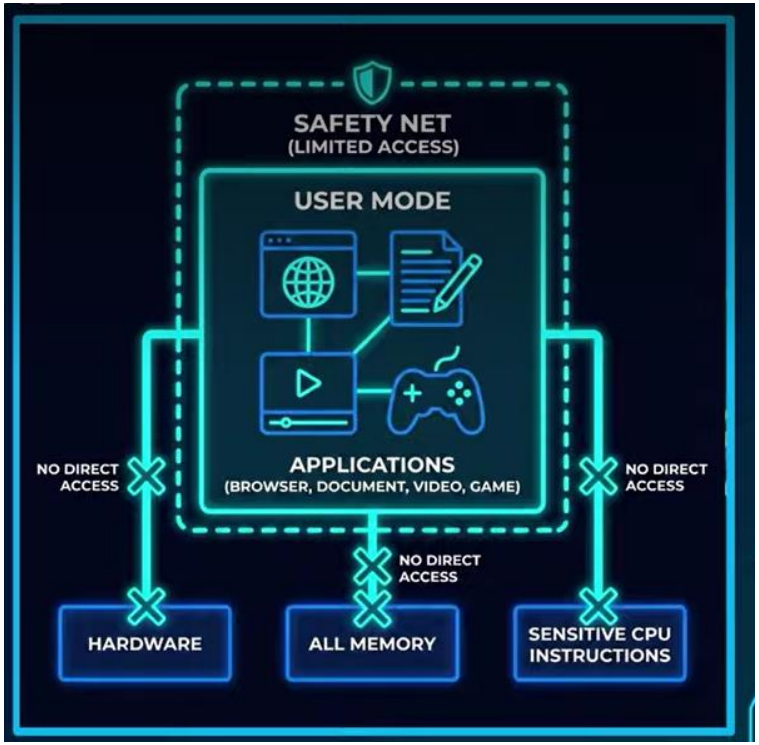
- Executing a **system call** ,execution mode switches from user mode to kernel mode.

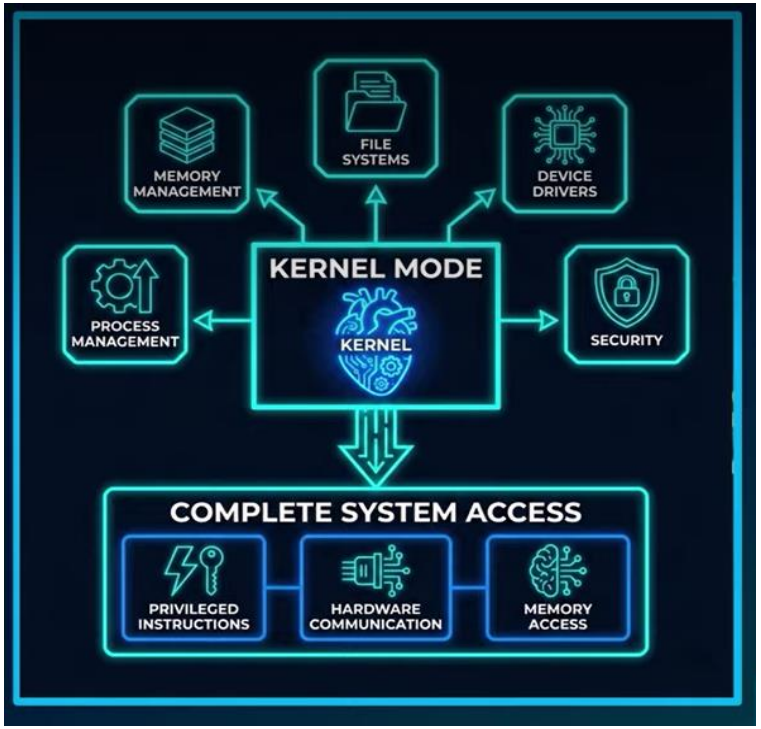
- Processes in user mode can’t access kernel instruction & data vice versa is not true.

- Some machine instruction are privileged & results in an error when executed in user mode.

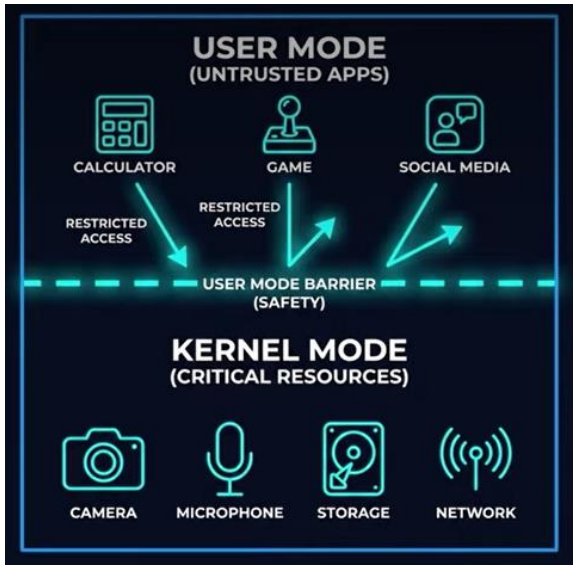


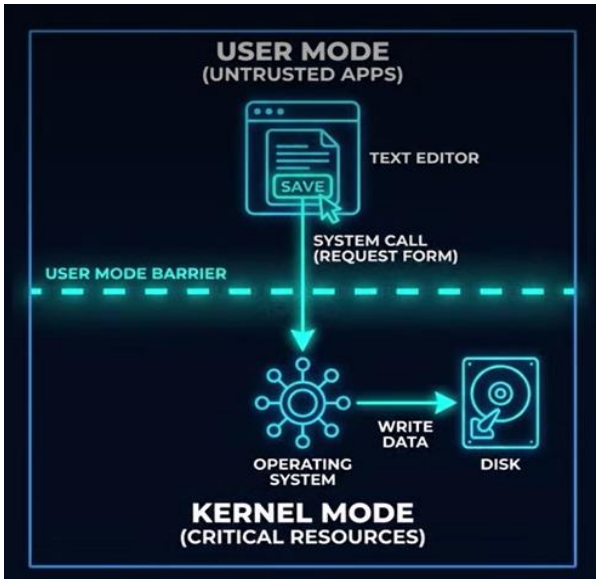
Process Execution levels…..



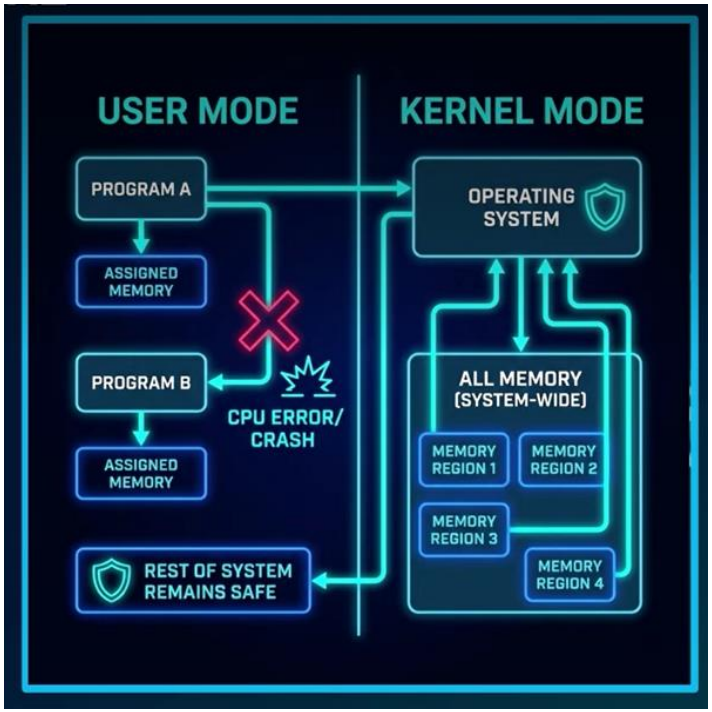


Process Execution levels…..





- **Memory Protection**
  - Each process gets its own memory space
  - User mode restricted to assigned memory
  - Illegal access causes process crash only
  - Kernel mode can access all memory



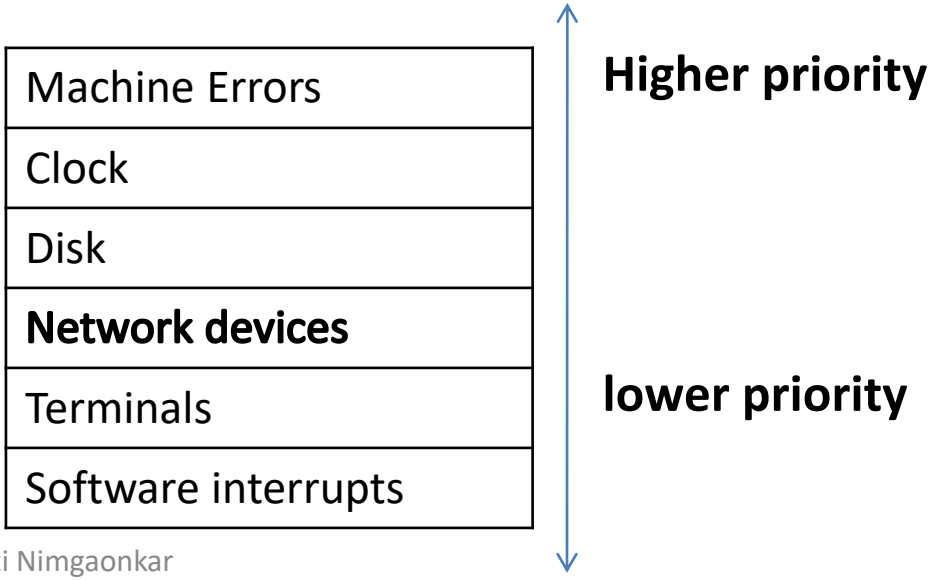
Interrupt and Exceptions…..

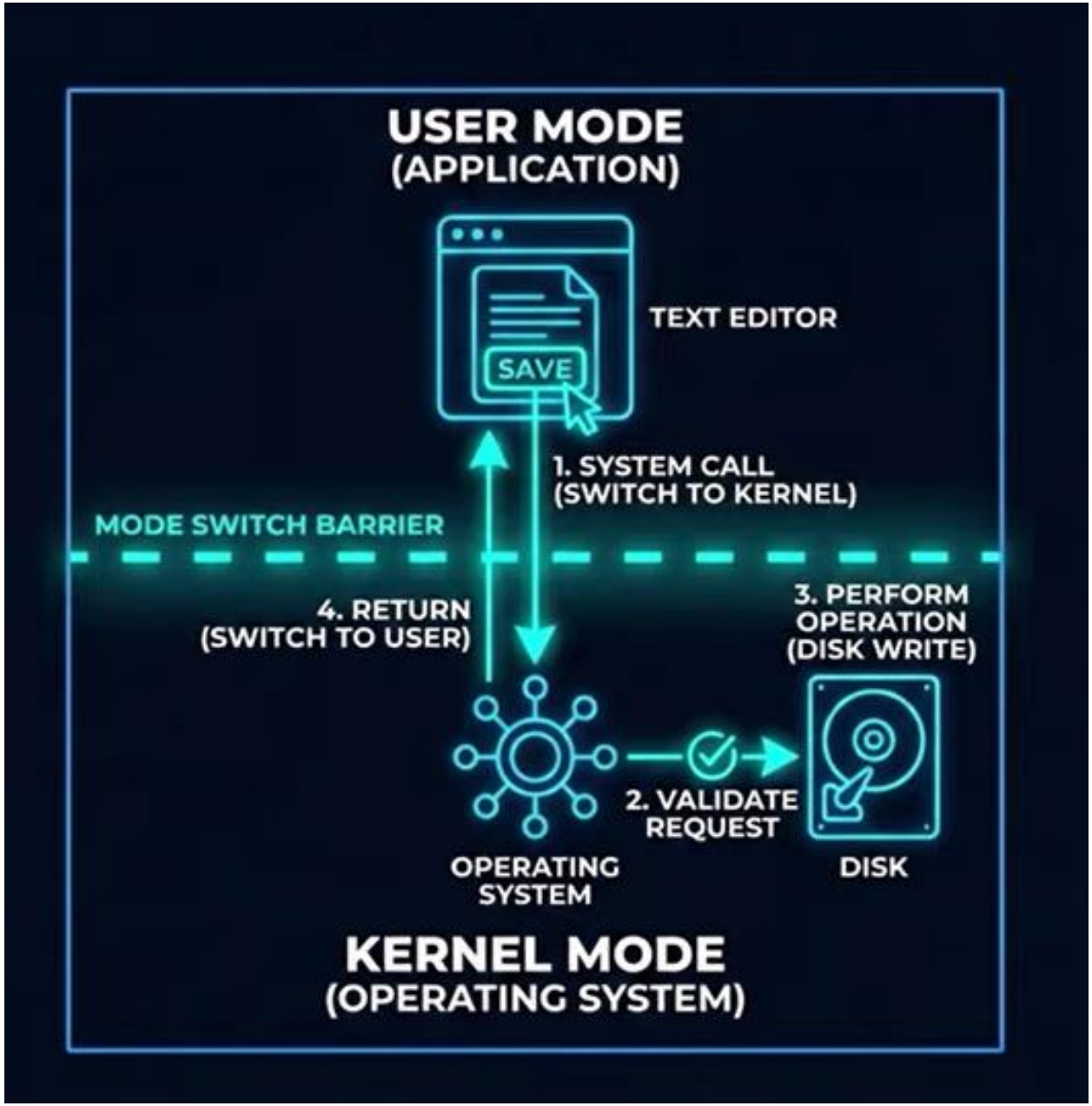
unexpected events which will disrupt the normal flow of execution *of instruction (that is currently executing by processor)*

- Hardware **interrupts** are called **Interrupts**.
- Software **interrupts** are called **Exceptions**.

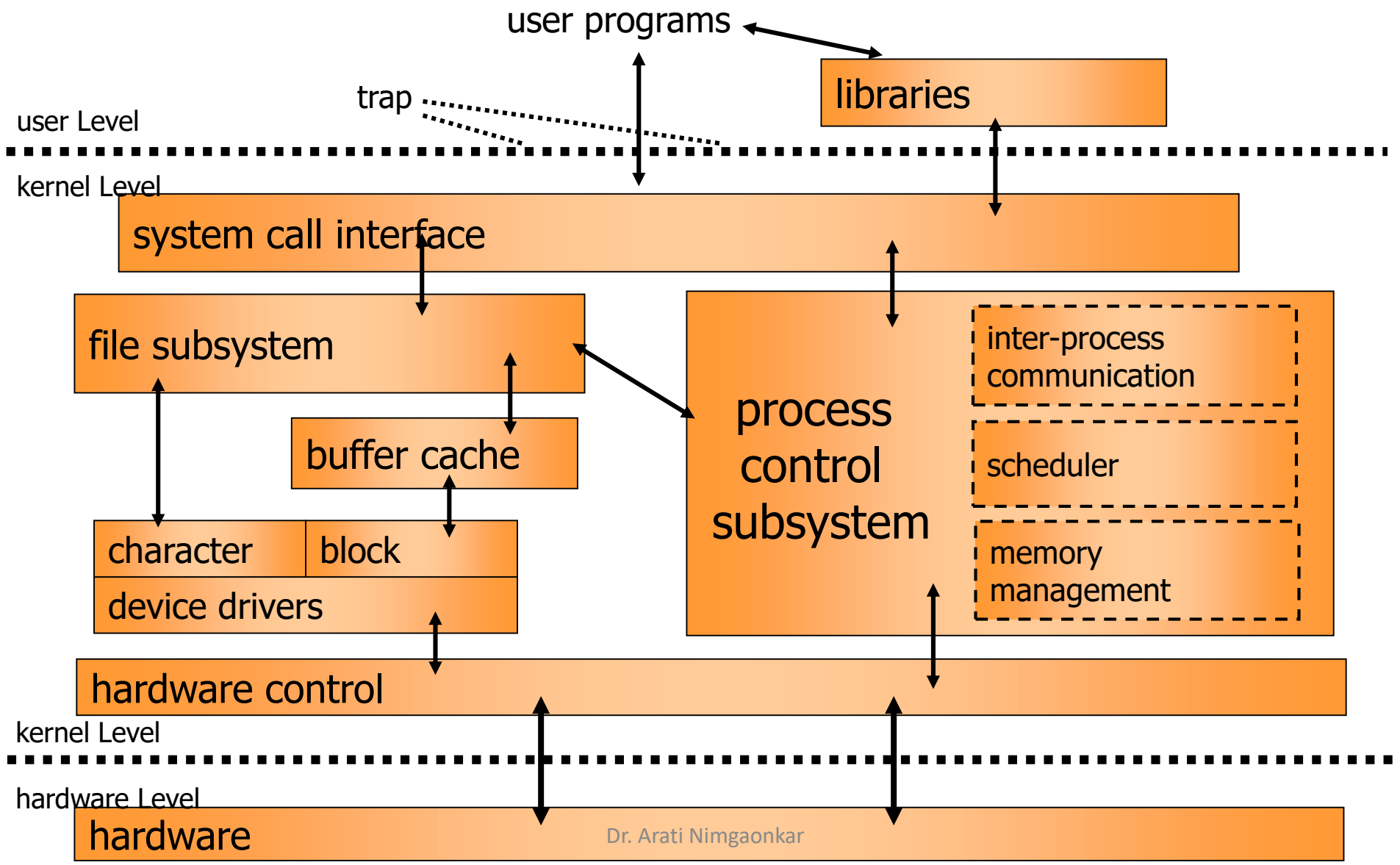
- Interrupts & exception
  - I/O peripherals or system clock, interrupts CPU.
  - Exception condition refers to unexpected events caused by process.

- Process execution levels



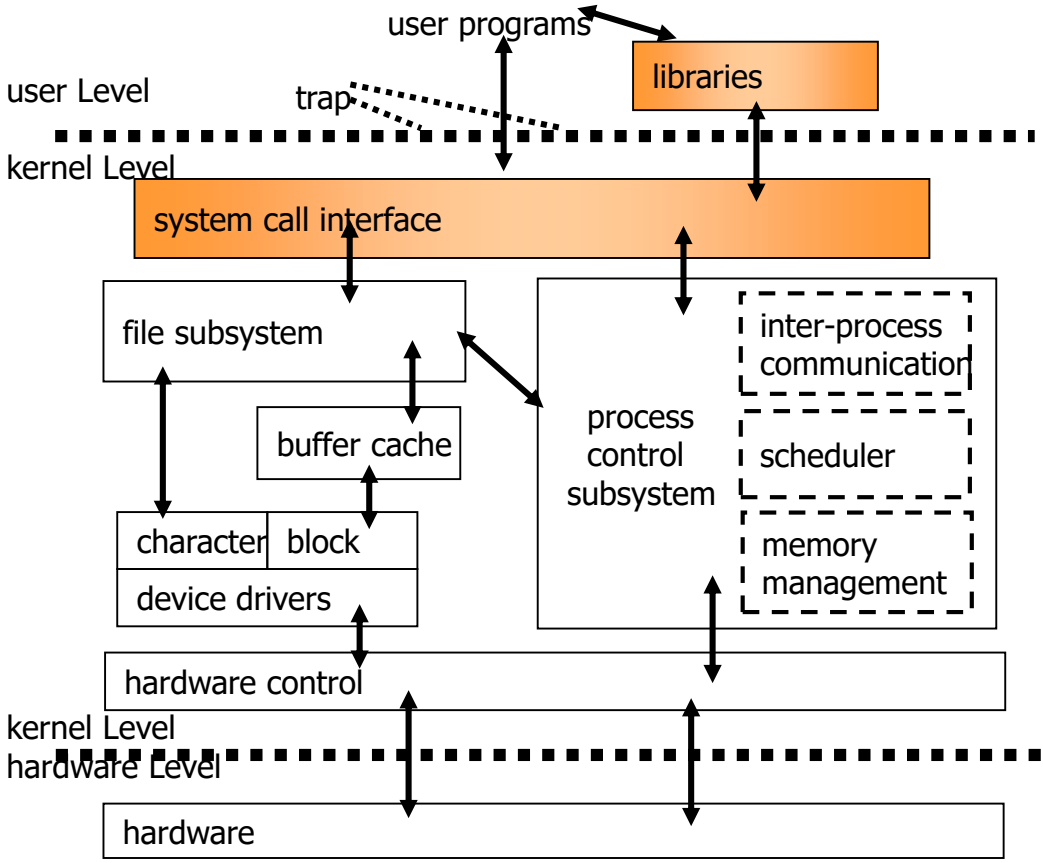

USER MODE
(APPLICATION)
SAVE
TEXT EDITOR
1. SYSTEM CALL
(SWITCH TO KERNEL)
MODE SWITCH BARRIER
4. RETURN
(SWITCH TO USER)
3. PERFORM
OPERATION
(DISK WRITE)
2. VALIDATE
REQUEST
OPERATING
SYSTEM
DISK
KERNEL MODE
(OPERATING SYSTEM)

Architecture of Unix



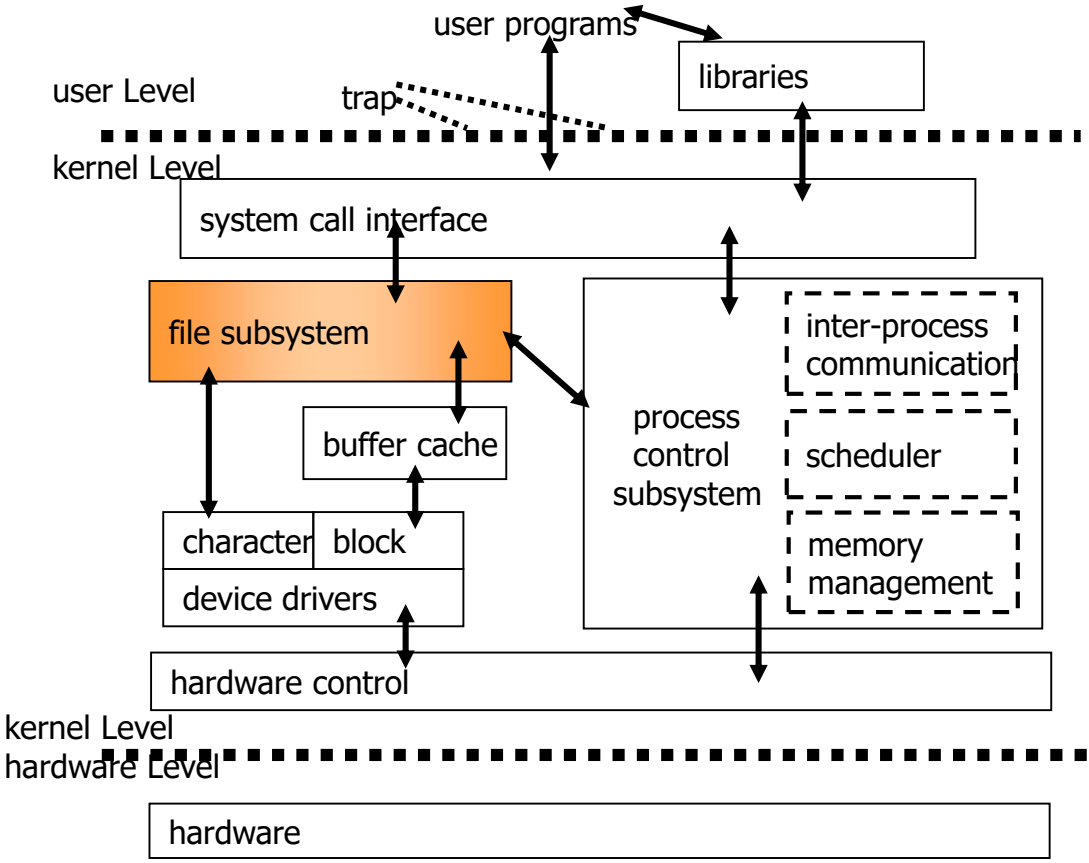
Architecture of Unix--libraries & System call

- System call & library represents the border between user & kernel level.
- System call contains ordinary function calls in C program,& Library maps these function calls to the primitives needed to enter the o.s.
- Assembly language program may invoke system calls directly.



Architecture of Unix—File subsystem

- Managing files
- Allocating file space
- Administering free space
- Controlling access to files
- Retrieving data for users
- Interact with set of system calls *-open, close, read, write, stat, chown, chmod …*



File subsystem –File System Layout



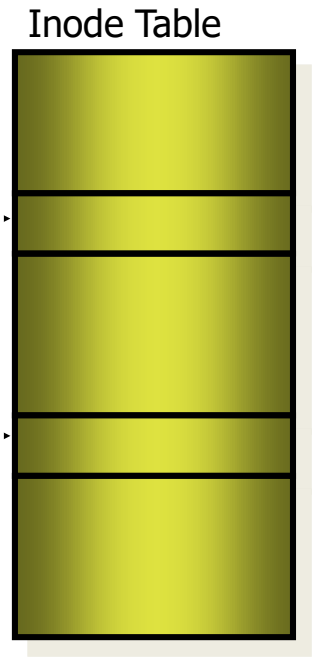
- boot block
  - Is needed to boot the system.
  - Contains boot strap code which initializes the operating system.
- super block
  - Describes the state of a file system.
    - How long it is?
    - How many files it can store?
    - Where to find the free space?
- inode list
  - a list of inodes
- data block
  - contain file data and administrative data

File subsystem data structures……

- Inode table – Maintains information related to all the files which exists.
- File Table – keep track of the data related to all open files.
- User File Descriptor Table – Maintains pointer which points to all open files.

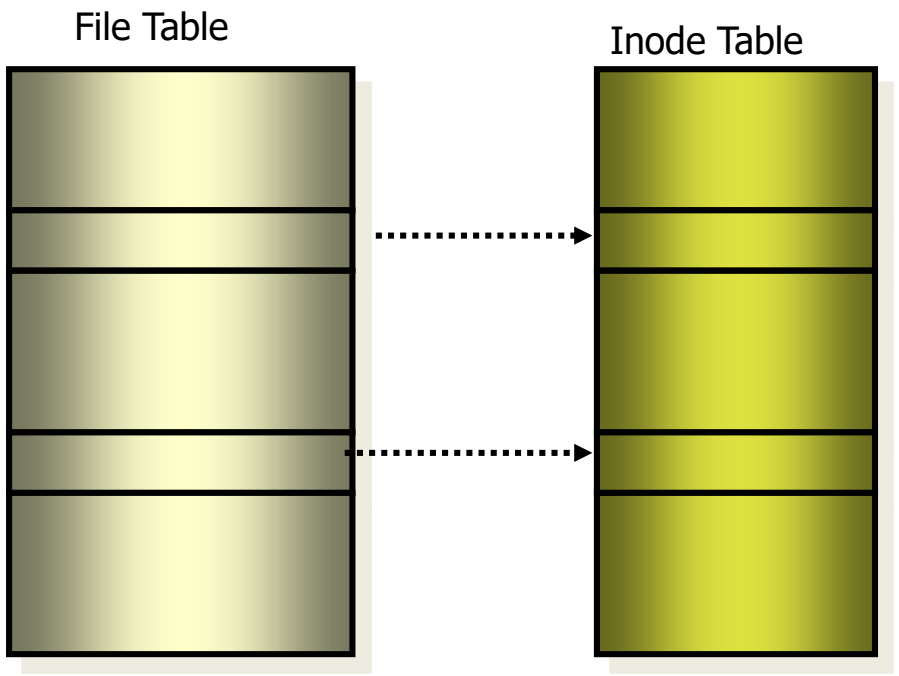
File subsystem

- Inode – Index Node
- Two types – Disk inode & In-core inode
- Maintains Information regarding inode of each file which is on disk.
- Maintains pointer to actual data of that inode(file)



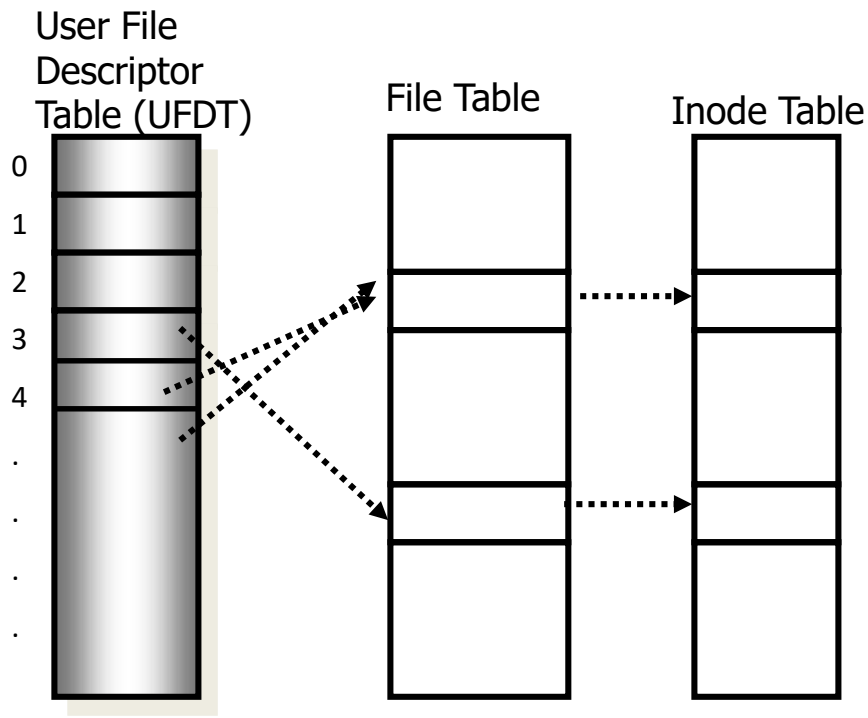
File subsystem

- File table keeps track of byte offset in the file for next read/write.
- Maintains pointer to inode table entry.

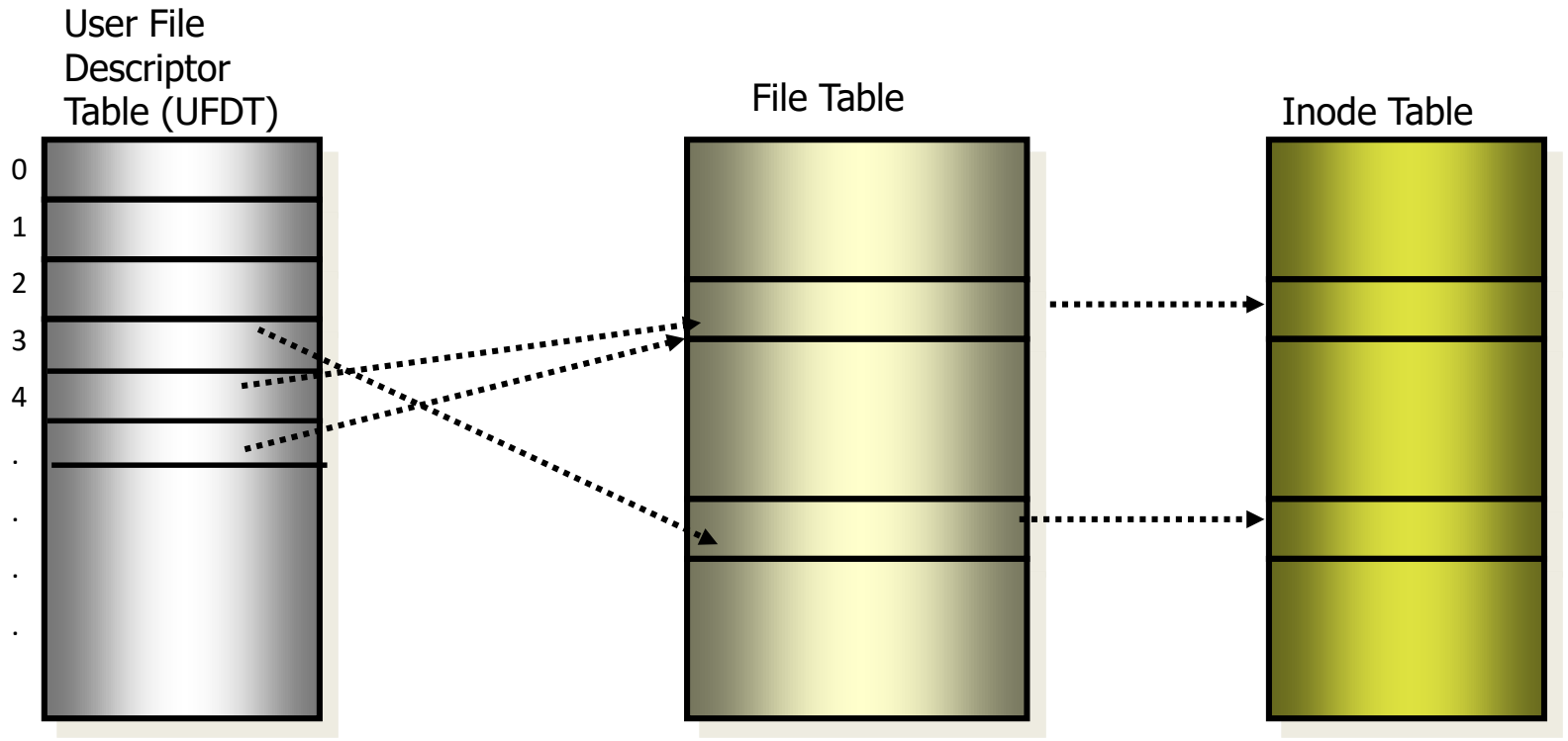


File subsystem –UFDT

- fd is return by file open &create system call.
- It is used as index into the UFDT which is used by system during read/write system call.
- Contains pointer to file table entry of that file.

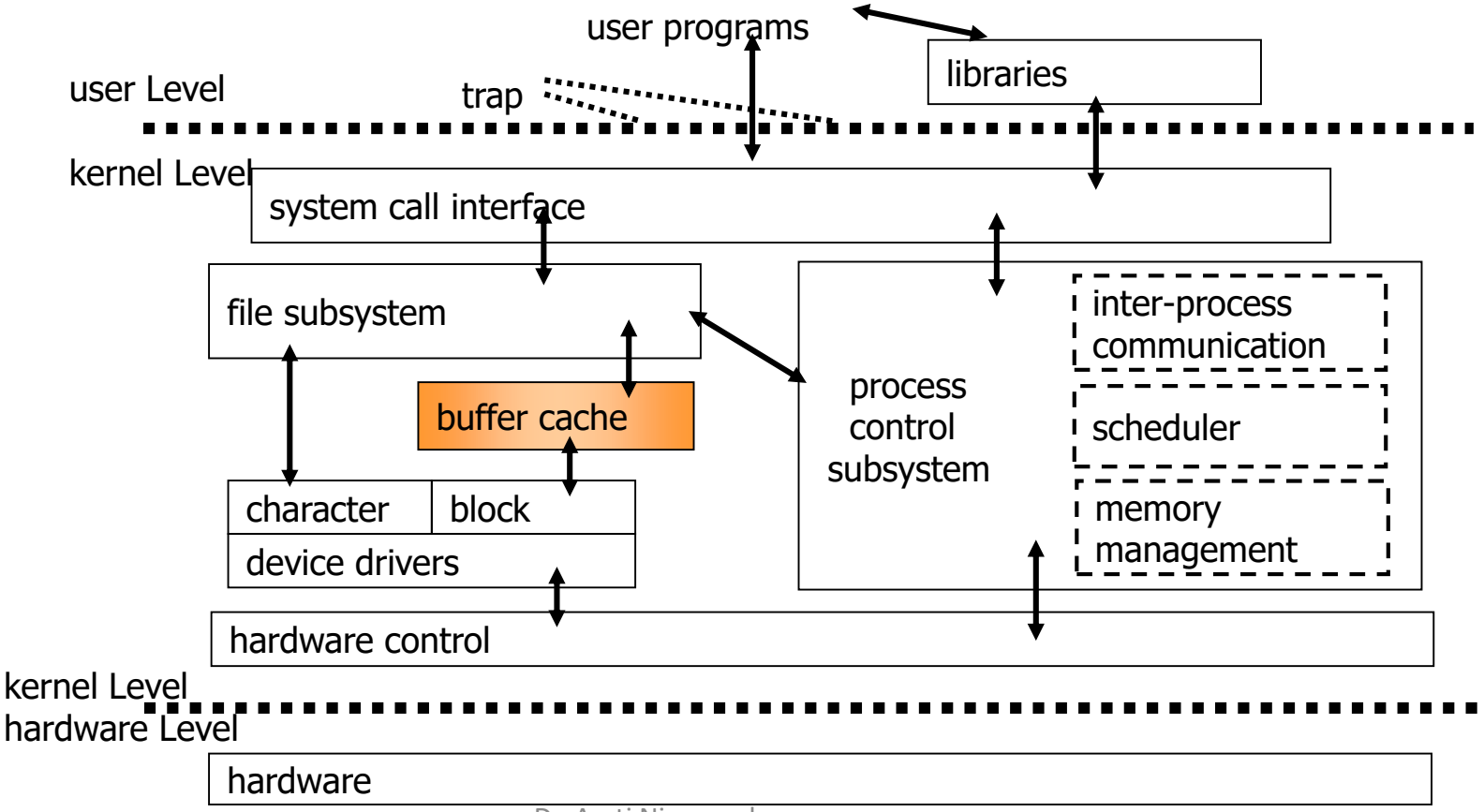


File subsystem



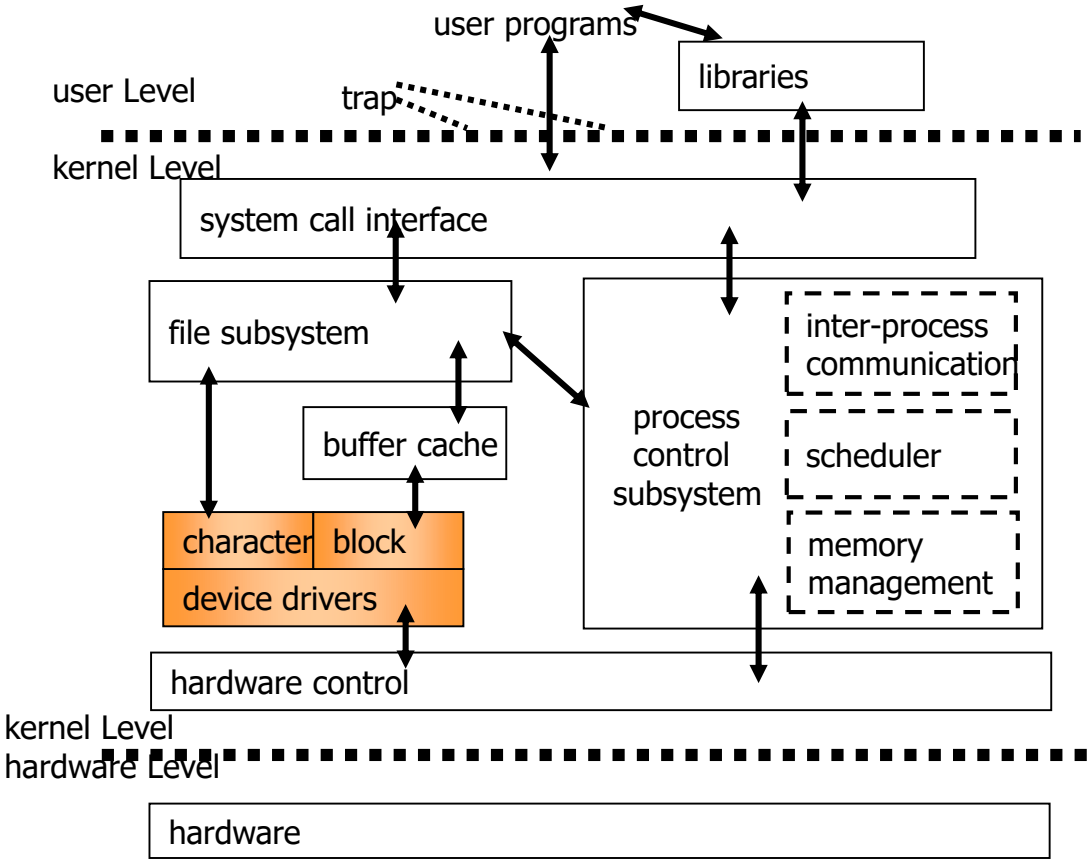
Architecture of Unix—Buffer cache

- Buffering mechanism is used to Interact with block I/O device drivers to initiate data transfer to and from kernel.



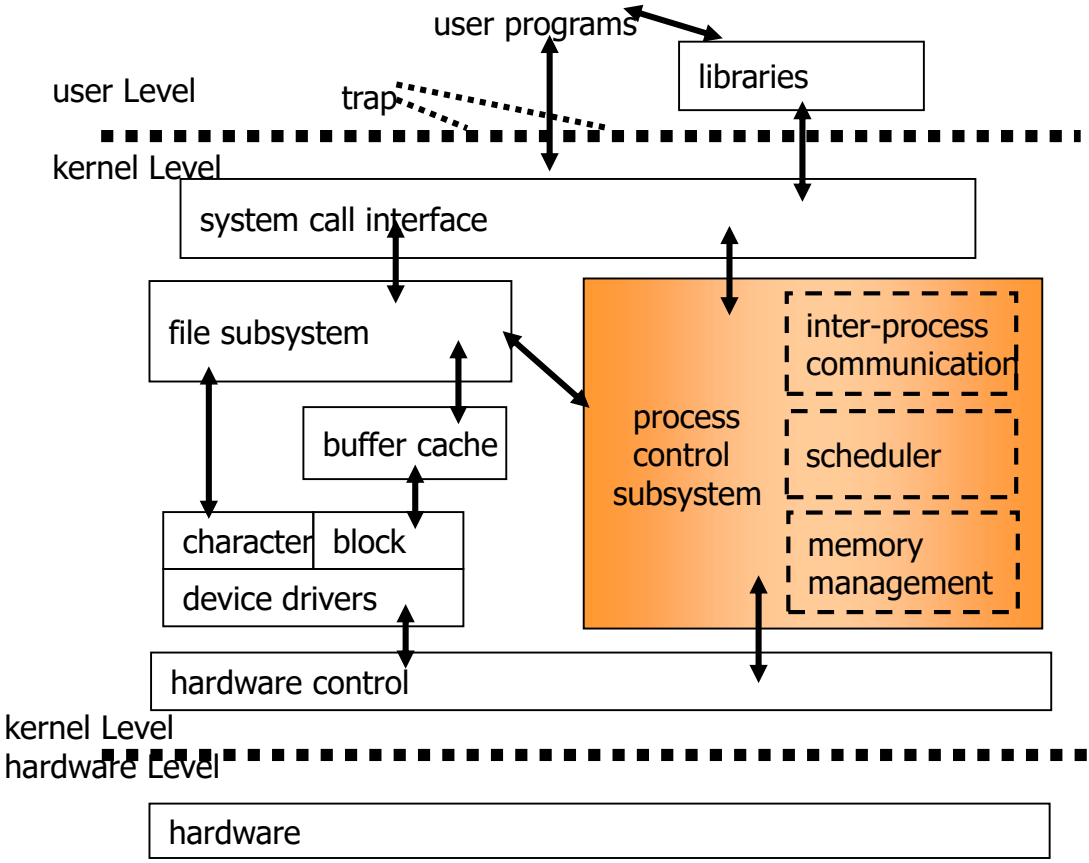
Architecture of Unix—Device drivers

- Device drivers control the operation of peripherals.
- Block devices are random access devices.
- Character devices are also called as raw devices.



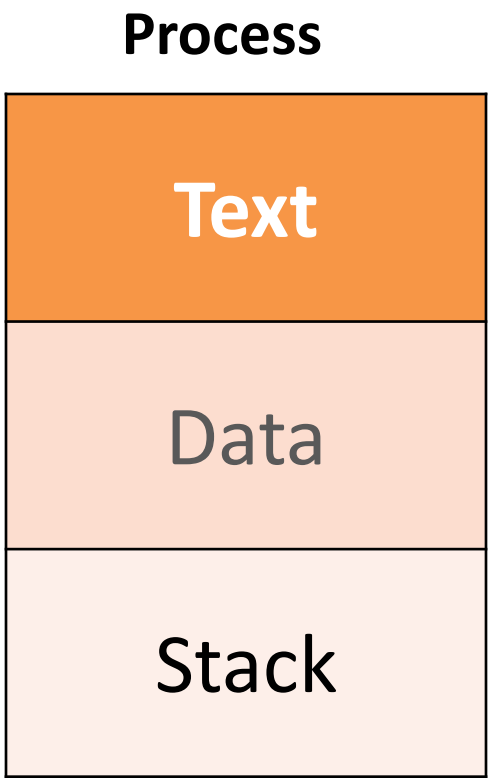
Architecture of Unix—Process control Subsystem

- Responsible for process synchronization.
- Interprocess communication (IPC)
- Memory management
- Process scheduling
- Interact with set of system calls
  - *fork, exec, exit, wait, brk, signal …*



Process subsystem--Region

- ***Text region*** contains executable code and read-only data, such as variables, and is typically marked read-only and executable..
- ***Data region*** contains initialized data, such as C variables with defined values, and is typically marked readable and writable.
- ***Stack region*** is auto created & size is dynamically adjusted



Process subsystem

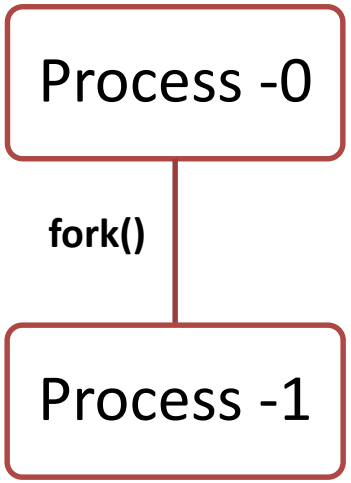
User compiles source code to create an executable file which contains……

- Set of ‘headers’
- Program text
- m/c lang. representation of data that has initial values when program starts execution, called as bss(block start by symbol).
- Other section information …..such as symbol table.

Process subsystem…process hierarchy

- ***Process -0*** is special process created by system boot. Also called as a ***Swapper process***

- ***Process-1*** is created by process-0 by calling fork(). It is also known as ***init process.***



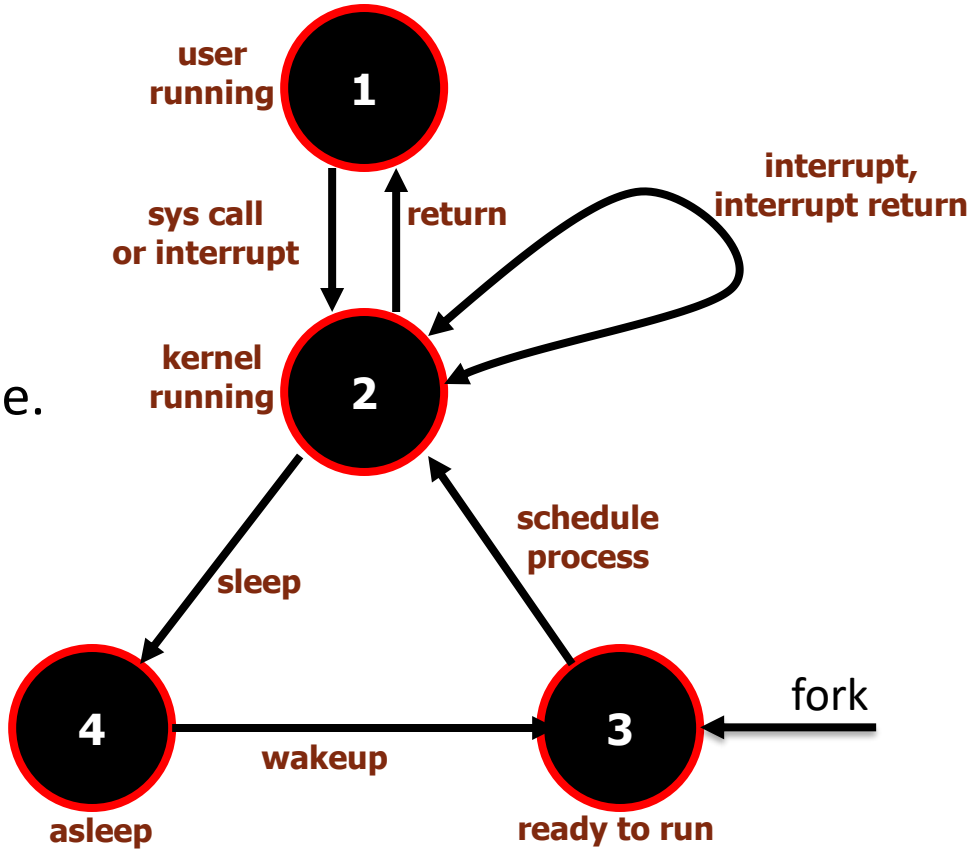
Process subsystem— Process State Transition

1. Process is executing in user mode
2. Process is executing in kernel mode
3. Process is not executing but is ready to run
4. Process is sleeping Process is ready to run ,but swapped on secondary device
5. Process is sleeping & swapped to secondary memory
6. Process returning from kernel to user, during kernel preempts it & does context switch to schedule another process.
7. Process is newly created

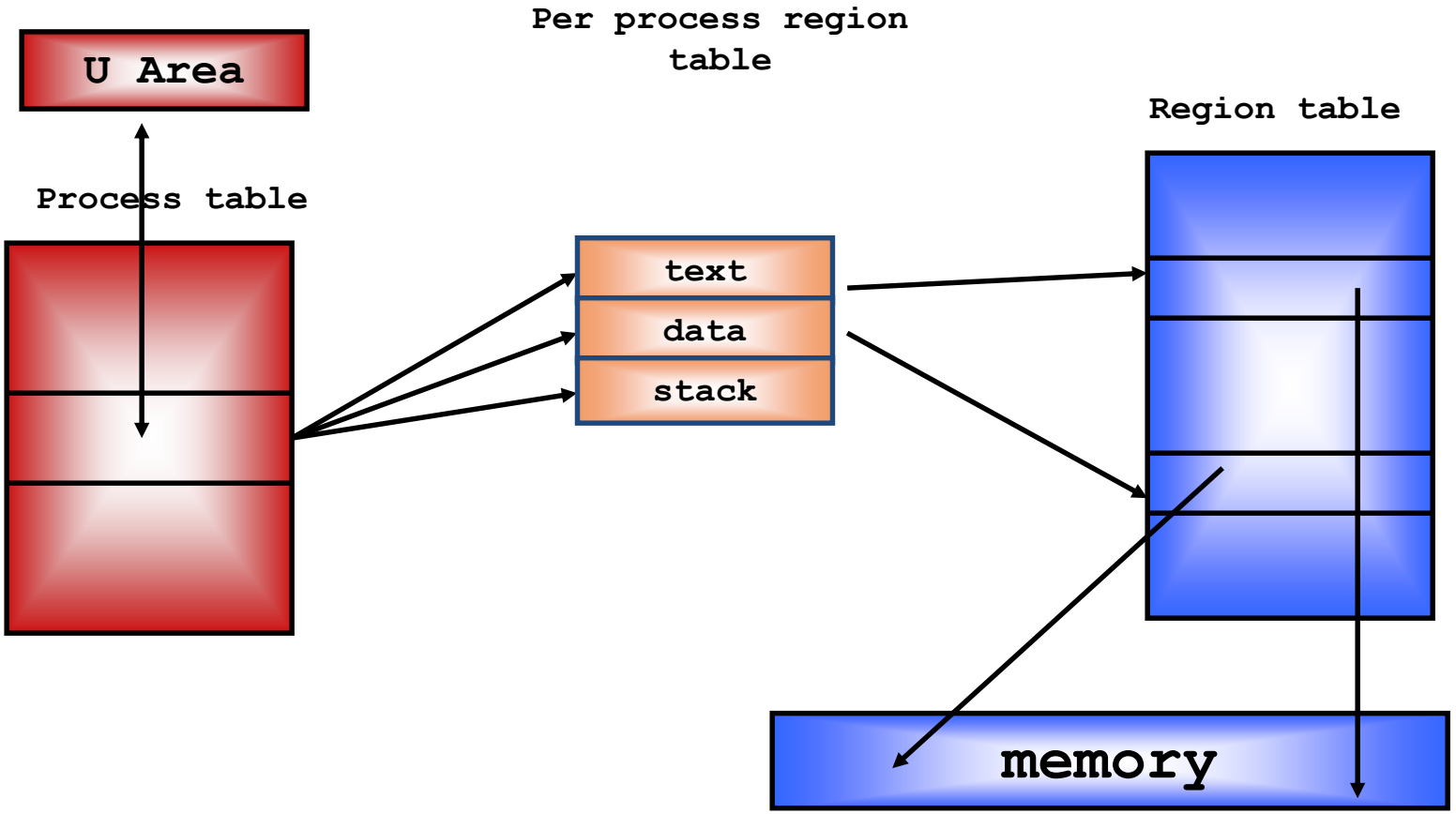
Process subsystem— Process State Transition

Process States

1. Process executing in user mode.
2. Process executing in kernel mode.
3. Process is in ready to run state.
4. Process is sleeping.



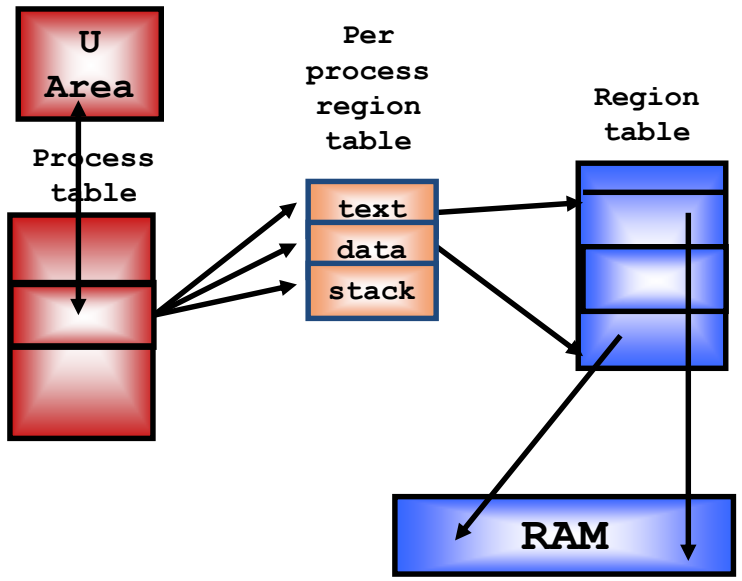
Process subsystem—Kernel Data Structures



Process subsystem—u-area

- Contains runtime information describing the process.

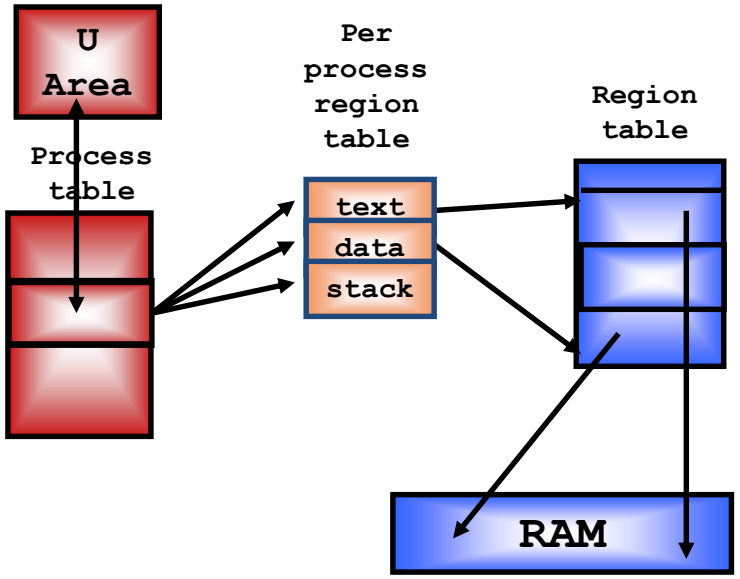
- Fields are …
  # Ptr to process table slot.
  # parameter to current system call.
  # fd's of open files
  # internal I/O parameter.
  # current dir. & current root.
  # process & the file size limits.
  # Different timers.



Process subsystem—Process Table

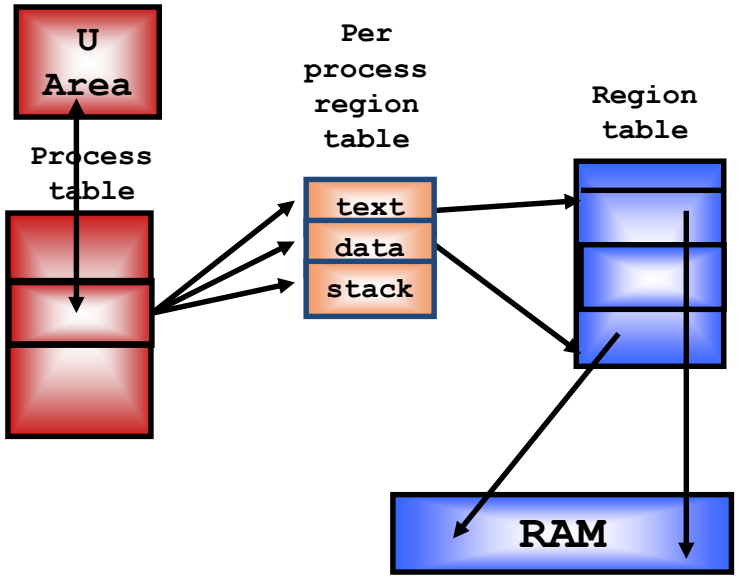
- Contains status & control information about process.

- Fields are …
  - # State
  - # process id, user id
  - # Event descriptor
  - # Scheduling parameters.
  - # Signal field
  - # Various timers.



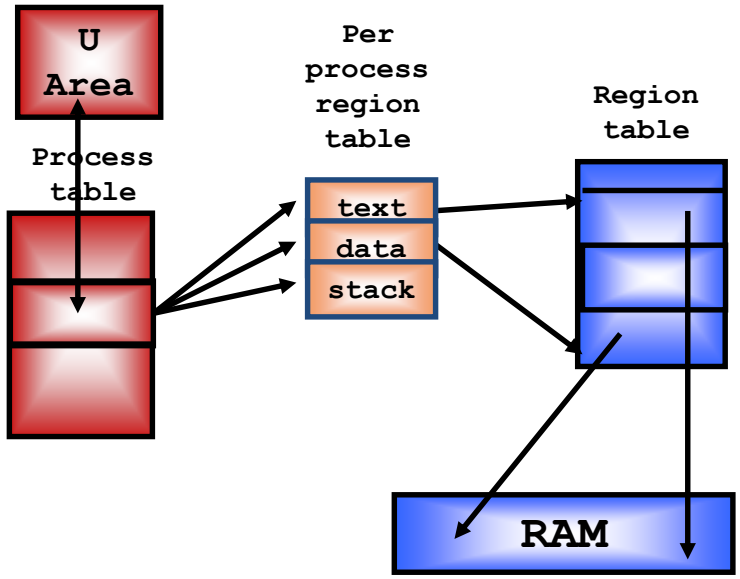
Process subsystem—Per process region table

- Contains pointer pointing to region table entry.
- This dynamic structure

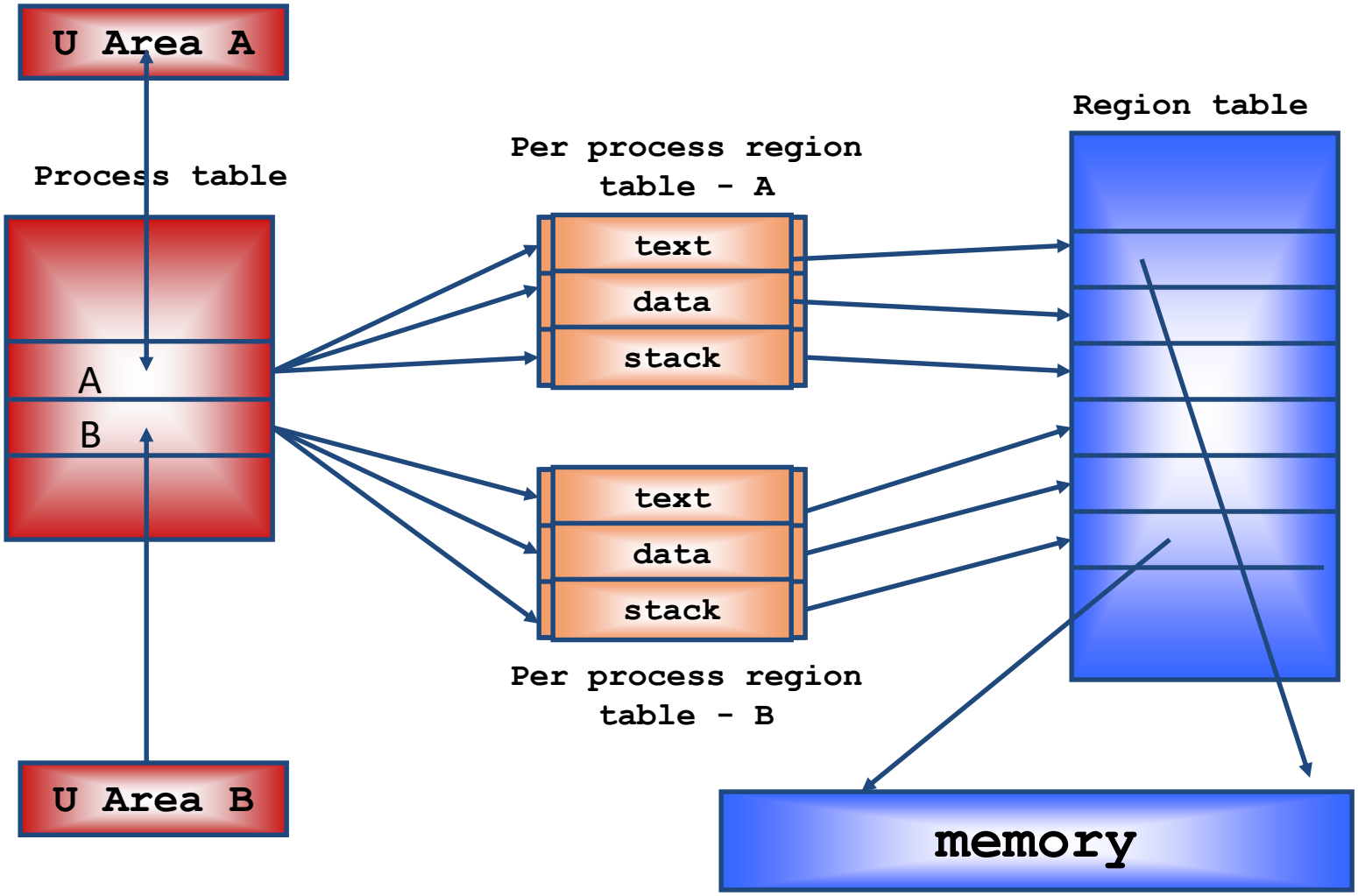


Process subsystem— Region table

- It is continuous area of process address space.

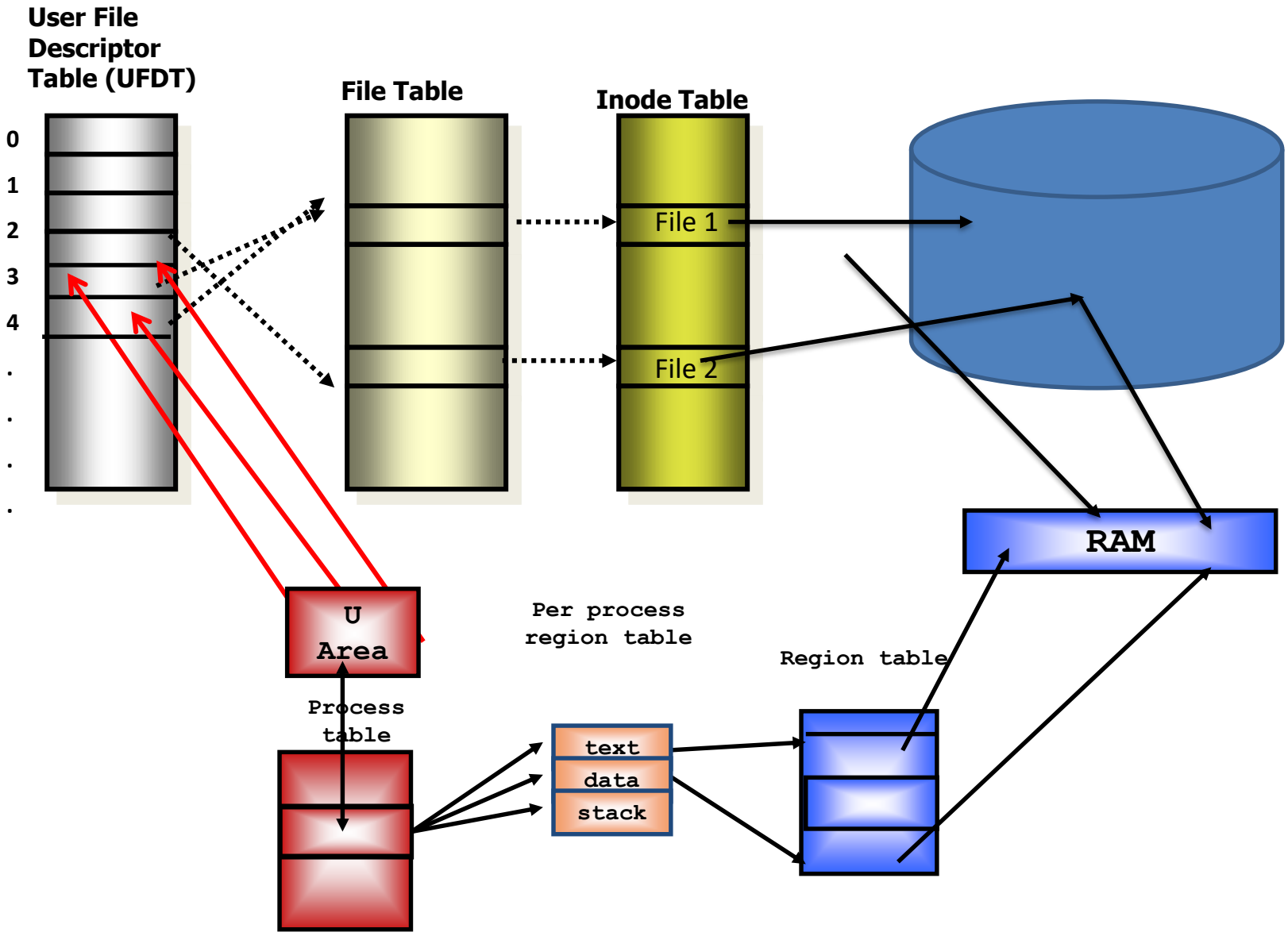


Process subsystem—Executing two processes



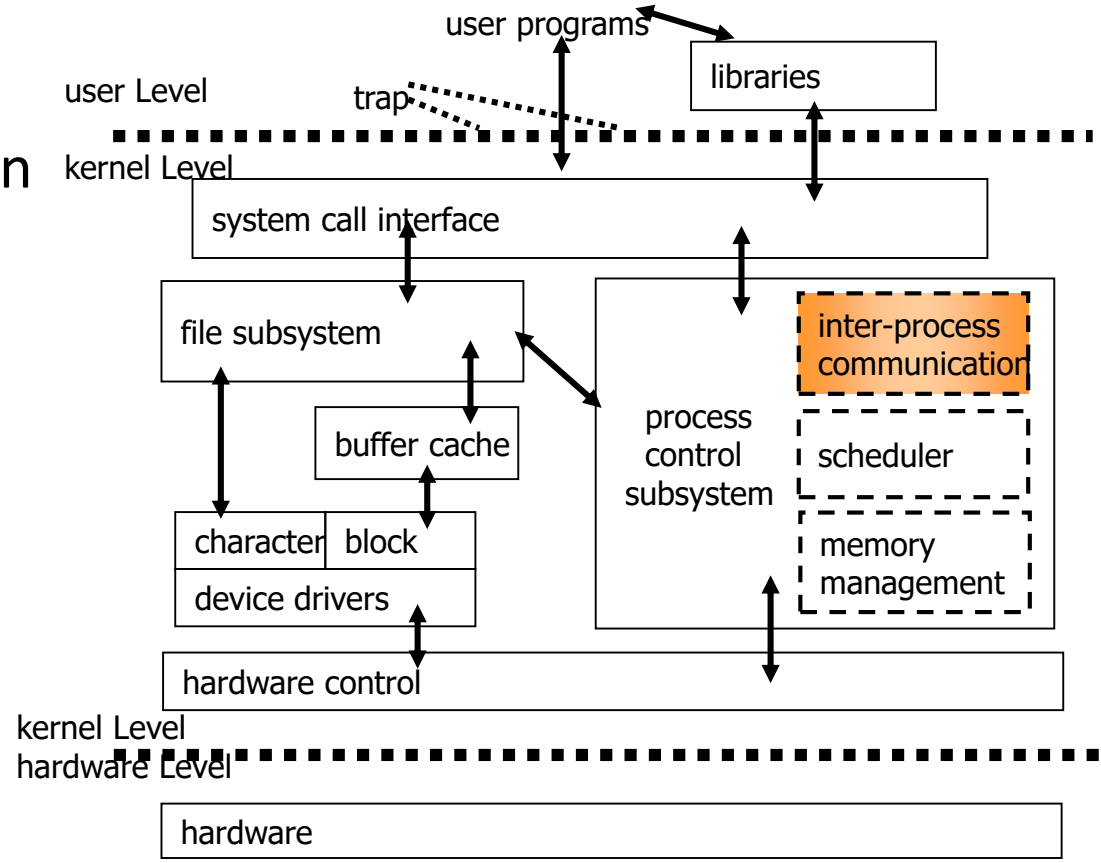
- Process A has Opend file1, file2

File subsystem and Process subsystem



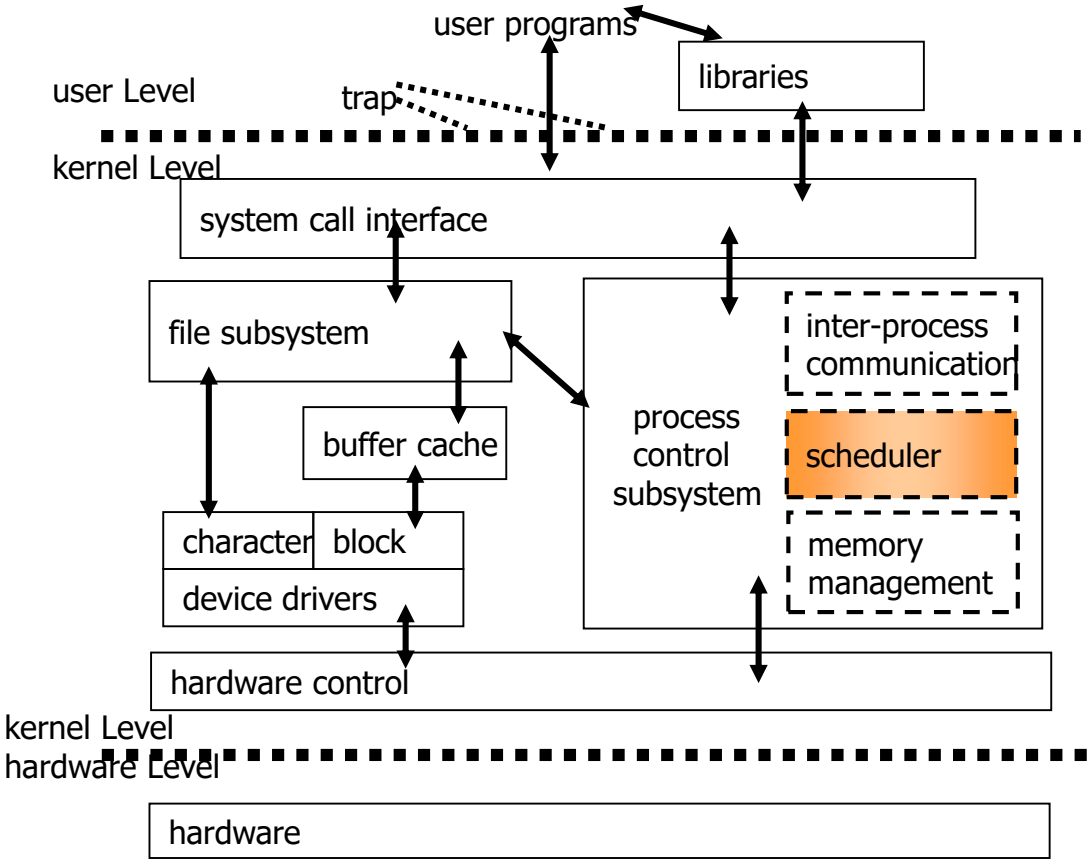
Architecture of Unix-Inter-Process Communication

- Inter-process communication (IPC) – used to synchronized transmission of messages between processes



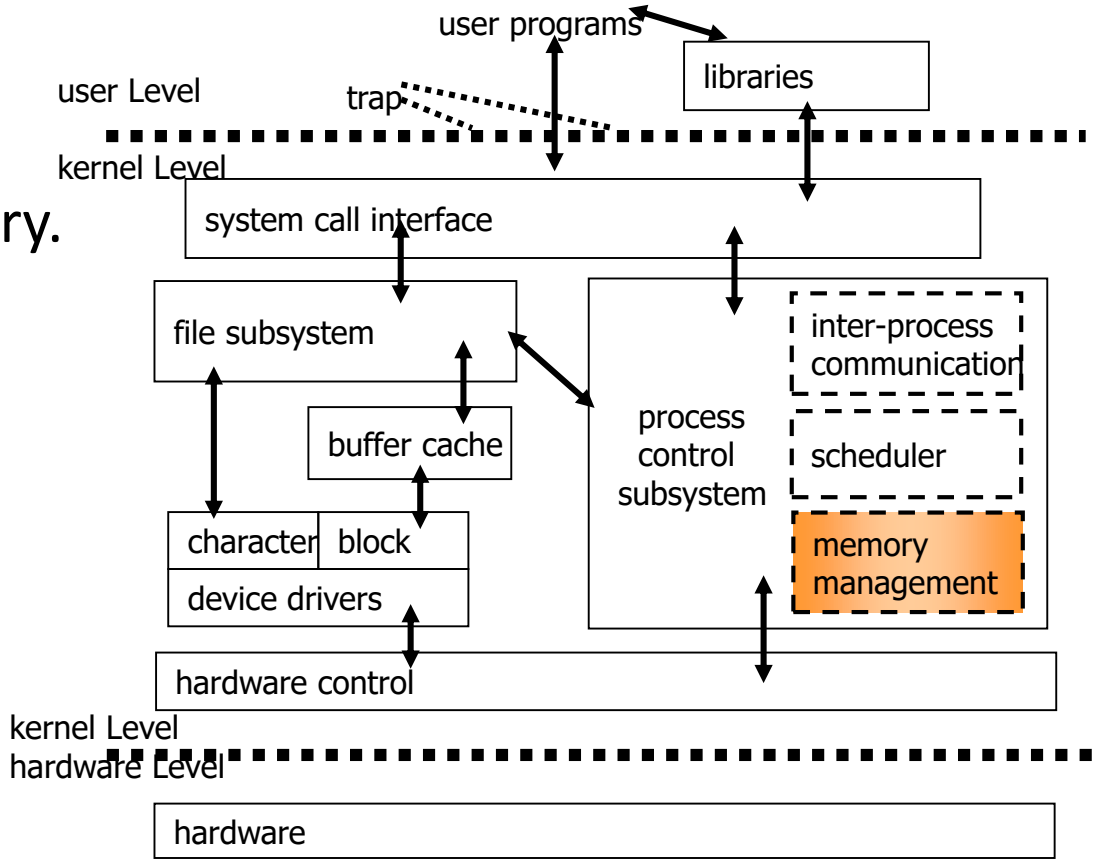
Architecture of Unix—Scheduler

- Used to allocate CPU to processes.



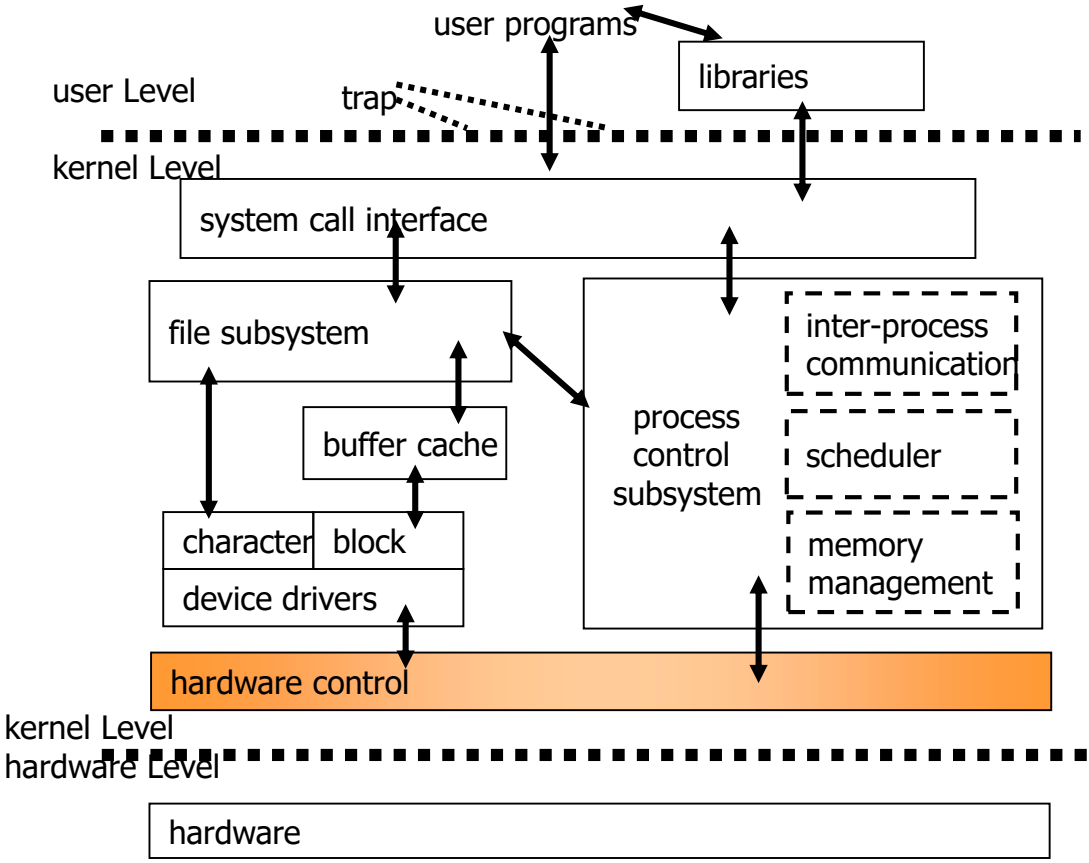
Architecture of Unix—Memory management

- Controls allocation of memory.
- Two policies swapping and demand paging is used.



Architecture of Unix—Hardware control

- Responsible for handling interrupts.
- Responsible for communicating with the machine.



Concept of Linux programming

Files and the File system

A file is referenced by an inode ,which is assigned a unique numerical value.

An inode stores metadata associated with a file, such as its modification timestamp, owner, type, length, and the location of the file's data—but no filename!!!

- Regular files
- Directories and links
- Hard links
- Symbolic links
- Special files

```
bob@ubuntu:~$ ls -il
total 56
26117 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Desktop
26121 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Documents
26118 drwxr-xr-x 2 bob  bob  4096 May  5 04:57 Downloads
22852 -rw-r--r-- 1 bob  bob  8445 Apr 22 06:14 examples.desktop
26122 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Music
26123 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Pictures
26120 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Public
31628 -rwxr-xr-x 1 root root   30 May  8 07:44 script.sh
26119 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Templates
21303 -rw-rw-r-- 1 bob  bob  1799 Apr 28 04:55 textfile.txt
21181 -rw-rw-r-- 1 bob  bob    13 Apr 28 04:53 textfile.txt~
26124 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Videos
bob@ubuntu:~$
```

Types of Files – Regular file

- Text files, data files, executable programs, shared libraries – stored as sequential and random access files.
- A regular file contains bytes of data, organized into a linear array called a byte stream.
- A single file can be opened more than once, by a different or even the same process.
- Each open instance of a file is given a unique file descriptor.

Types of Files – Directory file

- Directory contains named references to other files. Directories contain the names of files but not their contents.
- Conceptually, a directory is viewed like any normal file, with the difference that it contains only a mapping of names to inodes.
- The kernel directly uses this mapping to perform name-to-inode resolutions.
- Initially, there is only one directory on the disk, the root directory, denoted by the path /.
- A pathname that starts at the root directory is called an ***absolute pathname.***
- Some paths are provided relative to some other directory . These paths are called ***relative pathnames.***

Types of Files – Hard links

- Multiple file names can be there to the same inode , called as a ***hard link.***

- When multiple pathnames are there which points to the same data (inode),these are called as hard links.

- The hard links can be in the same directory, or in two or more different directories.

Types of Files – Symbolic links

- Hard links can’t span more than one file systems because an inode is meaningless outside of the inode's own file system.

- To allow links that can span more than one filesystem, Unix systems implement ***symbolic links***.

- A symlink has its own inode and data chunk, which contains the complete pathname of the linked-to file.

- **Symbolic links incur more overhead than hard links** because resolving a symbolic link effectively involves resolving two files: the symbolic link, and then the linked-to file.

Types of Files – Special purpose files

- Special files are kernel objects that are represented as files.

1. Device files –
   Random access devices.

2. Named pipes –
   Are an inter-process communication (IPC) mechanism

3. Unix domain sockets –
   Sockets are an advanced form of IPC, that allow for communication between two different processes, not only on the same machine, but on two different machines.

File Authorization

- Authorization is provided by users and groups.
- Each user is associated with a unique positive integer called the ***user ID (uid)***.
- Each ***process*** is in turn associated with exactly one uid.
- Usernames and their corresponding uids are stored in ***/etc/passwd***.
- The **uid 0** is associated with a special user known as **root**.
- Each user may belong to one or more groups.
- Every ***owner, group & other user*** has ***r/w/x*** permission.

File Authorization

- **Owner permissions** − The owner's permissions determine what actions the owner of the file can perform on the file.
- **Group permissions** − The group's permissions determine what actions a user, who is a member of the group that a file belongs to, can perform on the file.
- **Other (world) permissions** − The permissions for others indicate what action all other users can perform on the file.

```
bob@ubuntu:~$ ls -il
total 56
26117 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Desktop
26121 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Documents
26118 drwxr-xr-x 2 bob  bob  4096 May  5 04:57 Downloads
22852 -rw-r--r-- 1 bob  bob  8445 Apr 22 06:14 examples.desktop
26122 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Music
26123 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Pictures
26120 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Public
31628 -rwxr-xr-x 1 root root   30 May  8 07:44 script.sh
26119 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Templates
21303 -rw-rw-r-- 1 bob  bob  1799 Apr 28 04:55 textfile.txt
21181 -rw-rw-r-- 1 bob  bob    13 Apr 28 04:53 textfile.txt~
26124 drwxr-xr-x 2 bob  bob  4096 Apr 24 04:14 Videos
bob@ubuntu:~$
```

File Authorization

While using **ls -l** command, it displays various information related to file permission as follows −

$ls -l /home/amrood

-rwxr-xr--	1	amrood	users	1024	Nov 2 00:10	myfile
drwxr-xr---	1	amrood	users	1024	Nov 2 00:10	mydir

Here, the first column represents different access modes, i.e., the permission associated with a file or a directory.

The permissions are broken into groups of threes, and each position in the group denotes a specific permission, in this order: read (r), write (w), execute (x) −

File Authorization

Each permission is assigned a value, as the following table shows, and the total of each set of permissions provides a number for that set.

Number	Octal Permission Representation	Ref
0	No permission	---
1	Execute permission	--x
2	Write permission	-w-
3	Execute and write permission: 1 execute + 2 write = 3	-wx
4	Read permission	r--
5	Read and execute permission: 4 read + 1 execute = 5	r-x
6	Read and write permission: 4 read + 2 write = 6	rw-
7	All permissions: 4 read + 2 write + 1 execute = 7	rwx